Informe del proyecto

Clasificación de lesiones cutáneas pigmentadas con redes neuronales convolucionales

Comparación experimental sobre HAM10000

Milton Alejandro Cuervo Ramirez
Ingeniería de Sistemas · Universidad de Antioquia
Fundamentos de Deep Learning
Docentes: Raúl Ramos y Julián Arias
13 de noviembre de 2026

Resumen

Este proyecto estudia la clasificación de lesiones cutáneas mediante una red convolucional entrenada desde cero y tres arquitecturas preentrenadas en ImageNet. Para comparar su comportamiento se utilizaron las 10.015 imágenes de HAM10000, distribuidas por lesión entre entrenamiento, validación y prueba. La selección de los modelos se realizó con la balanced accuracy de validación, mientras que las 1.494 imágenes de prueba se reservaron para la evaluación final. En la tarea binaria, la CNN con pérdida ponderada obtuvo una balanced accuracy de **0,7865** y una sensibilidad de **94,03 %** para el grupo maligno o premaligno. ResNet50, elegida entre las redes preentrenadas, alcanzó 0,7585 y 77,04 %, respectivamente. Al ampliar el problema a siete categorías, ResNet50 con ajuste fino obtuvo una balanced accuracy de **0,7123** y un F1 macro de **0,5814**, aunque la sensibilidad para identificar específicamente melanoma fue de **61,50 %**. Estos resultados describen el desempeño observado en el experimento y deben interpretarse dentro de su alcance académico.

Objetivo y alcance

El objetivo fue establecer cómo cambian los resultados al pasar de una CNN sencilla a VGG16, ResNet50 y EfficientNet-B0, y determinar qué dificultades persisten cuando se intenta distinguir las siete categorías originales. La comparación también permitió examinar el efecto de ponderar la pérdida y de ajustar una parte de la red preentrenada, teniendo en cuenta las limitaciones de memoria del equipo disponible.

Tarea	Definición y criterio
Binaria	Positiva: mel, bcc y akiec; benigna: bkl, df, nv y vasc.
Multiclase	Siete diagnósticos originales; una categoría por imagen.
Selección	Máxima balanced accuracy de validación; test consultado después de fijar la selección.

En la tarea binaria, una predicción positiva indica que la imagen pertenece al grupo de lesiones malignas o premalignas. Esta decisión agrupa tres diagnósticos distintos y, por tanto, no identifica por sí sola un melanoma.

1. Datos, particiones y preprocesado

Se trabajó con HAM10000, un conjunto de 10.015 imágenes dermatoscópicas agrupadas en siete categorías [1], obtenido a través de su distribución en Kaggle [5]. Sus 7.470 identificadores de lesión reflejan que algunas lesiones cuentan con más de una imagen. Para evitar que fotografías de una misma lesión aparecieran en conjuntos distintos, la división se realizó por lesion_id, con semilla 42 y una proporción aproximada de 70 % para entrenamiento, 15 % para validación y 15 % para prueba.

Clase	Descripción	Entr.	Val.	Prueba	Total
akiec	Queratosis actínica / carcinoma intraepitelial	207	57	63	327
bcc	Carcinoma basocelular	381	65	68	514
bkl	Lesión similar a queratosis benigna	741	206	152	1099
df	Dermatofibroma	89	19	7	115
mel	Melanoma	770	156	187	1113
nv	Nevus melanocítico	4718	991	996	6705
vasc	Lesión vascular	96	25	21	142
Total	Imágenes	7002	1519	1494	10015
Lesiones	lesion_id únicos	5229	1120	1121	7470

La distribución presenta un desbalance marcado: los nevus reúnen 6.705 imágenes, equivalentes al 66,95 % del conjunto, mientras que solo hay 115 dermatofibromas. En la tarea binaria, las lesiones malignas o premalignas suman 1.954 imágenes (19,51 %); de ellas, 318 quedaron en prueba, junto con 1.176 imágenes benignas. Esta diferencia de frecuencias hizo necesario complementar la exactitud global con medidas que valoraran el comportamiento de cada clase.

Controles para evitar contaminación

Antes del entrenamiento se revisó que los metadatos correspondieran a las imágenes disponibles y que los archivos originales pudieran leerse correctamente. También se verificó que las particiones no compartieran lesiones ni duplicados exactos, estos últimos mediante SHA-256. Se conservaron todas las imágenes y se calcularon los pesos de clase únicamente con los datos de entrenamiento. El control se hizo por lesión; dado que no se contó con una identificación de pacientes para este propósito, no puede asegurarse independencia entre pacientes.

Transformaciones ejecutadas

La preparación consistió en convertir las imágenes a RGB y redimensionarlas a 224 × 224 píxeles mediante interpolación bilineal. El resultado se guardó en una caché PNG para reducir el trabajo repetido durante cada época, sin modificar los originales. Durante el entrenamiento se añadieron volteos horizontales y verticales aleatorios y rotaciones de hasta 20°. La normalización siguió los valores de ImageNet, con medias (0,485; 0,456; 0,406) y desviaciones (0,229; 0,224; 0,225). En validación y prueba se mantuvo una transformación determinista, sin aumentos aleatorios.

La licencia declarada por la copia de Kaggle es CC-BY-NC-SA-4.0 [5]. El uso realizado es académico y debe conservar la atribución y las condiciones de esa distribución.

Obtención y disponibilidad de los datos

Los datos no se incluyen en el repositorio por su tamaño. Para obtenerlos se utiliza la distribución **Skin Cancer MNIST: HAM10000** de Kaggle [5], identificada como **kmader/skin-cancer-mnist-ham10000**. El siguiente procedimiento deja los archivos en las ubicaciones que esperan los notebooks.

1. Definir la carpeta de trabajo

Abrir Jupyter desde la raíz del proyecto, donde están los cinco notebooks numerados, y ejecutar **01_exploracion_de_datos.ipynb** desde el comienzo. La variable PROJECT toma el directorio de trabajo actual y todas las rutas se construyen a partir de él. Para comprobar la ubicación, la sección «Datos locales» muestra en pantalla la carpeta de destino. En Colab, el directorio de trabajo habitual es /content; los notebooks posteriores necesitan acceder a los archivos generados en ese mismo espacio de trabajo.

2. Descargar desde el notebook 01

Se necesita una cuenta de Kaggle y un API token obtenido en su configuración. Si no existe **data/ham10000/HAM10000_metadata.csv**, la celda de «Datos locales» instala el cliente de Kaggle y solicita el token mediante una entrada oculta, salvo que ya esté disponible en la variable de entorno KAGGLE_API_TOKEN o en el archivo local data/.kaggle/access_token. El token se introduce únicamente en ese mecanismo de autenticación; no debe escribirse en el código ni publicarse en GitHub.

La celda descarga el archivo comprimido, extrae su contenido en **data/ham10000/** y elimina el ZIP después de la extracción. Si los datos ya están disponibles, reutiliza los archivos locales. Como alternativa, se puede descargar el ZIP desde la página de Kaggle y extraerlo manualmente en esa carpeta. El CSV debe quedar directamente dentro de data/ham10000/; las imágenes JPG pueden permanecer en sus subcarpetas, porque el notebook las busca de forma recursiva.

3. Verificar los archivos y preparar las entradas

Continuar el notebook 01 hasta «Metadatos y rutas» para comprobar las 10.015 filas y su correspondencia con las imágenes. Después ejecutar **02_preprocesado.ipynb**, que crea o conserva las particiones en **data/splits/**, verifica los originales y genera las imágenes RGB de 224 × 224 en **data/processed_224/**. Esta última carpeta es la entrada de imágenes utilizada por los procesos de entrenamiento.

4. Mantener disponibles los datos durante el entrenamiento

Ejecutar los notebooks 03, 04 y 05 en orden y desde la misma raíz, conservando las carpetas data/ y results/. Los entrenamientos leen los manifiestos de data/splits/ y las imágenes de data/processed_224/; el notebook 05 también necesita la selección producida por el 04 en results/. Si se cambia de equipo o se reinicia un entorno temporal de Colab, hay que restaurar esas carpetas o volver a ejecutar los pasos previos. Abrir un notebook desde GitHub no transfiere los datos ni los archivos generados por los demás.

2. Diseño experimental ejecutado

Experimento	Arquitectura y entrenamiento
CNN desde cero	Tres bloques Conv 3×3–BatchNorm–ReLU–MaxPool, con 16/32/64 canales; pooling global, dropout 0,3 y salida de dos clases. 23.938 parámetros. Dos variantes: pérdida sin pesos y ponderada; 10 épocas cada una.
Transferencia binaria	VGG16 [2], ResNet50 [3] y EfficientNet-B0 [4], pesos IMAGENET1K_V1. Backbone congelado; cabeza de dos clases entrenada durante 5 épocas. VGG16 usa pooling global y una cabeza compacta.
ResNet50 multiclase	Inicialización ImageNet y cabeza nueva de siete clases: 8 épocas. Se recuperó el mejor checkpoint de cabeza y se ajustaron layer4 y la cabeza durante 5 épocas adicionales.

Configuración común

Todos los experimentos se ejecutaron con el optimizador Adam, lotes de ocho imágenes, semilla 42 y precisión FP32. Se utilizó una tasa de aprendizaje de 10^{-3} para la CNN y los clasificadores nuevos, y de 10^{-4} durante el ajuste fino, sin decaimiento de pesos. El número de épocas se fijó antes de cada ejecución y no se emplearon cambios automáticos de la tasa de aprendizaje ni parada temprana. Las partes congeladas de las redes permanecieron en modo de evaluación para conservar sus parámetros y estadísticas.

Como función de pérdida se empleó la entropía cruzada. En los experimentos ponderados, cada clase recibió el peso $\mathbf{w(c) = N / [K \cdot n(c)]}$, donde N es el número de imágenes de entrenamiento, K el número de clases y n(c) la frecuencia de la clase correspondiente. De esta manera, los errores en las clases menos frecuentes tuvieron mayor peso durante la optimización. Esta estrategia se aplicó sin un muestreador ponderado.

Selección y evaluación

Al finalizar cada época se registraron las pérdidas de entrenamiento y validación, junto con la balanced accuracy y el F1 macro de validación. Se conservó el checkpoint con mayor balanced accuracy y, cuando hubo empate, el primero que alcanzó ese valor. El mismo criterio permitió elegir la arquitectura preentrenada. Una vez fijadas estas decisiones, los tres modelos finales se evaluaron en prueba mediante la clase de mayor probabilidad, sin ajustar umbrales a partir de sus etiquetas.

La **balanced accuracy** se calculó como el promedio de la sensibilidad de las clases, mientras que el **F1 macro** promedia el F1 de cada una. La **AUC-ROC** resume la capacidad de ordenar ejemplos a lo largo de distintos umbrales; para siete clases se empleó el esquema one-vs-rest (OvR) con promedio macro. También se registró la tasa de falsos negativos (**FNR**), equivalente a uno menos la sensibilidad. Estas medidas permiten examinar aspectos distintos del rendimiento y no deben interpretarse como equivalentes.

Equipo y diferencias frente a la propuesta

El entrenamiento se realizó en una NVIDIA GTX 1050 de 3 GB, con Python 3.12, PyTorch 2.6.0, CUDA 11.8 y torchvision 0.21.0. La memoria disponible obligó a reducir el tamaño de lote de 32, previsto en la propuesta, a ocho imágenes. La versión final comparó la pérdida sin ponderación y la pérdida con pesos de clase; el muestreador WeightedRandomSampler y las variaciones de brillo y color quedaron fuera de la implementación, por lo que no se presentan resultados sobre esas alternativas.

3. Entrenamiento y selección por validación

Experimento	Épocas	Mejor	BAcc	F1 macro	Minutos
CNN sin pesos	10	10	0.6955	0.6877	10.01
CNN con pesos	10	10	0.7492	0.6085	9.71
VGG16 · cabeza	5	4	0.7458	0.6117	15.40
ResNet50 · cabeza	5	3	0.7927	0.6901	9.38
EfficientNet-B0 · cabeza	5	4	0.7920	0.7054	5.74
ResNet50 · 7 clases, cabeza	8	3	0.5707	0.3942	15.21
ResNet50 · ajuste fino	5	5	0.7281	0.6154	13.61

La tabla presenta las métricas del checkpoint seleccionado por balanced accuracy (BAcc). La columna «Mejor» indica su época dentro de cada etapa. El tiempo corresponde a la suma de entrenamiento y validación registrada en las épocas, sin incluir descarga, preparación de datos ni inicialización de los modelos.

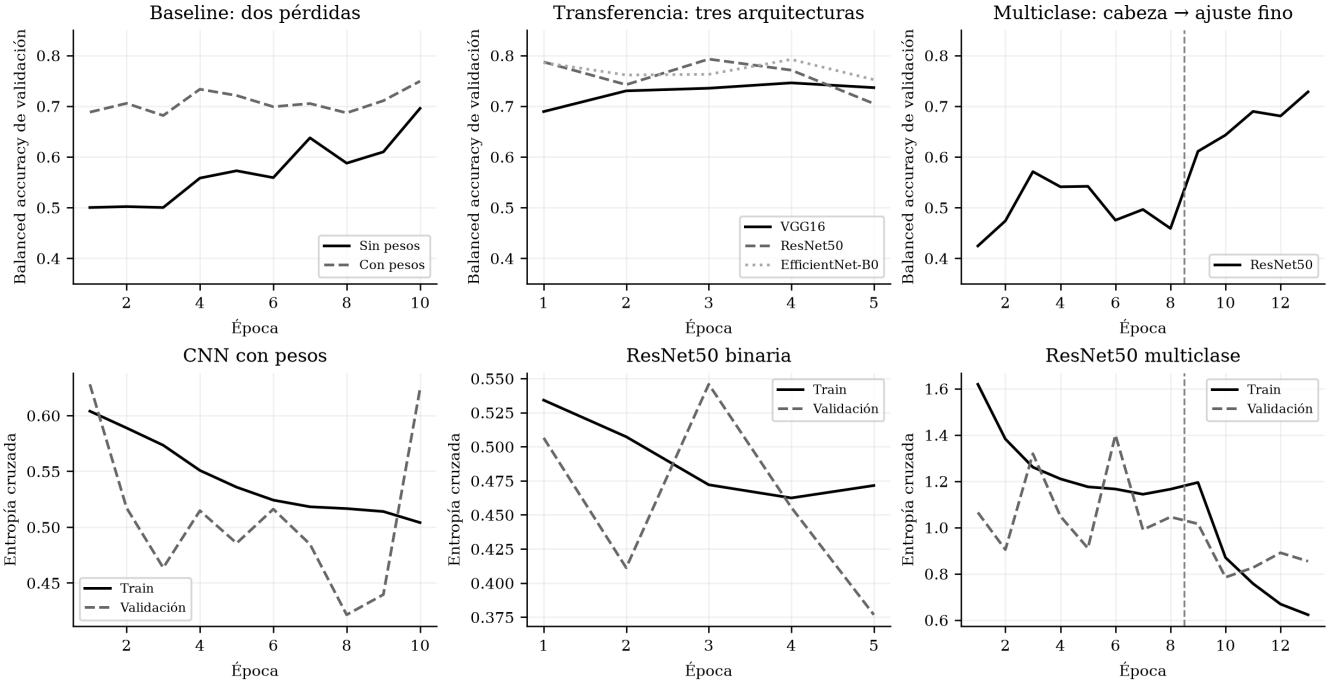


Figura 1. Evolución de la balanced accuracy de validación y de las pérdidas durante el entrenamiento. La línea vertical de las gráficas multiclase marca el paso de las ocho épocas del clasificador a las cinco de ajuste fino, que comenzaron desde el mejor checkpoint de la primera etapa.

En la CNN, introducir pesos de clase aumentó la balanced accuracy de validación de 0,6955 a 0,7492, aunque el F1 macro del checkpoint seleccionado fue menor. Entre las redes preentrenadas se eligió ResNet50 por su valor de 0,7927, frente a 0,7920 de EfficientNet-B0. La diferencia, de apenas 0,066 puntos porcentuales, fue suficiente para aplicar la regla de selección establecida, pero no permite concluir que una arquitectura sea consistentemente superior a la otra.

En el problema multiclase, entrenar el último bloque de ResNet50 junto con el clasificador elevó la mejor balanced accuracy de validación de 0,5707 a 0,7281. La menor pérdida de validación se observó antes de alcanzar ese máximo, lo que muestra que ambos indicadores no evolucionaron de la misma manera. Por esta razón se mantuvo el criterio definido de antemano, en lugar de seleccionar el modelo por la última época o por la menor pérdida.

4. Test reservado: clasificación binaria

La evaluación binaria se realizó con los dos modelos seleccionados y las mismas 1.494 imágenes de prueba. En ambos casos, la categoría positiva reunió mel, bcc y akiec. Para comprobar los resultados, las métricas se calcularon nuevamente a partir de los archivos de predicciones y se contrastaron con los reportes guardados durante la ejecución.

Modelo	BAcc	F1 macro	AUC-ROC	Exactitud
CNN ponderada	0.7865	0.6687	0.8649	0.6981
ResNet50	0.7585	0.6973	0.8483	0.7517

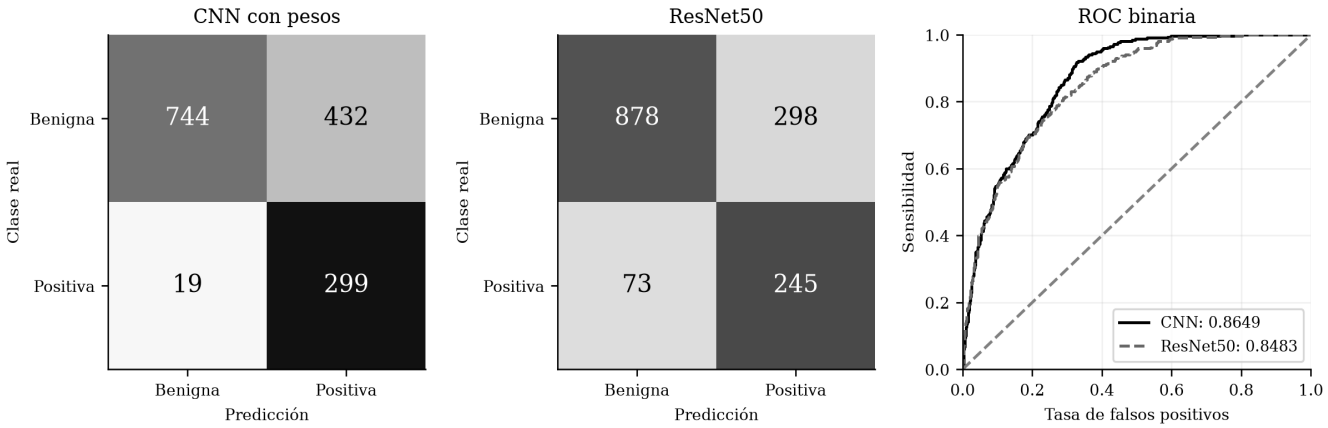


Figura 2. Matrices de confusión y curvas ROC de la tarea binaria. Las filas corresponden a la clase real y las columnas a la predicha; los números muestran cantidades de imágenes y la intensidad de gris representa su proporción dentro de cada fila. La categoría positiva incluye lesiones malignas o premalignas.

Indicador del grupo positivo	CNN ponderada	ResNet50
Sensibilidad / recall	94,03 % (299/318)	77,04 % (245/318)
Especificidad	63,27 % (744/1.176)	74,66 % (878/1.176)
Precisión positiva	40,90 %	45,12 %
Falsos negativos / FNR	19 / 5,97 %	73 / 22,96 %
Falsos positivos	432	298

Los resultados muestran un compromiso entre detectar más lesiones positivas y reducir las falsas alarmas. La CNN obtuvo mayor sensibilidad, balanced accuracy y AUC, pero produjo 134 falsos positivos más que ResNet50. Esta última mejoró la exactitud global y el F1 macro, aunque clasificó como benignas 54 imágenes positivas adicionales. La comparación permite entender las diferencias entre los modelos sin modificar la selección que ya se había realizado con validación.

Melanoma dentro de la tarea binaria

Al examinar únicamente las 187 imágenes de melanoma del conjunto de prueba, la CNN incluyó 175 en el grupo positivo (**93,58 %**), mientras que ResNet50 incluyó 135 (**72,19 %**). La CNN superó así la meta exploratoria del 85 % de sensibilidad planteada para esta detección agrupada. Sin embargo, este resultado debe distinguirse del reconocimiento específico de melanoma entre siete diagnósticos, que se analiza a continuación.

5. Test reservado: siete categorías

Después del ajuste fino, ResNet50 obtuvo una **balanced accuracy de 0,7123**, un **F1 macro de 0,5814** y una exactitud global de 0,7309. La AUC OvR macro fue de 0,9429; su valor no se compara directamente con la AUC binaria porque corresponde a una evaluación distinta. El desglose por clase permite apreciar diferencias que quedan ocultas en los promedios.

Clase	Precisión	Recall	F1	FNR	n
akiec	0.5500	0.6984	0.6154	0.3016	63
bcc	0.5811	0.6324	0.6056	0.3676	68
bkl	0.4459	0.6513	0.5294	0.3487	152
df	0.1250	0.7143	0.2128	0.2857	7
mel	0.4792	0.6150	0.5386	0.3850	187
nv	0.9516	0.7701	0.8513	0.2299	996
vasc	0.5938	0.9048	0.7170	0.0952	21

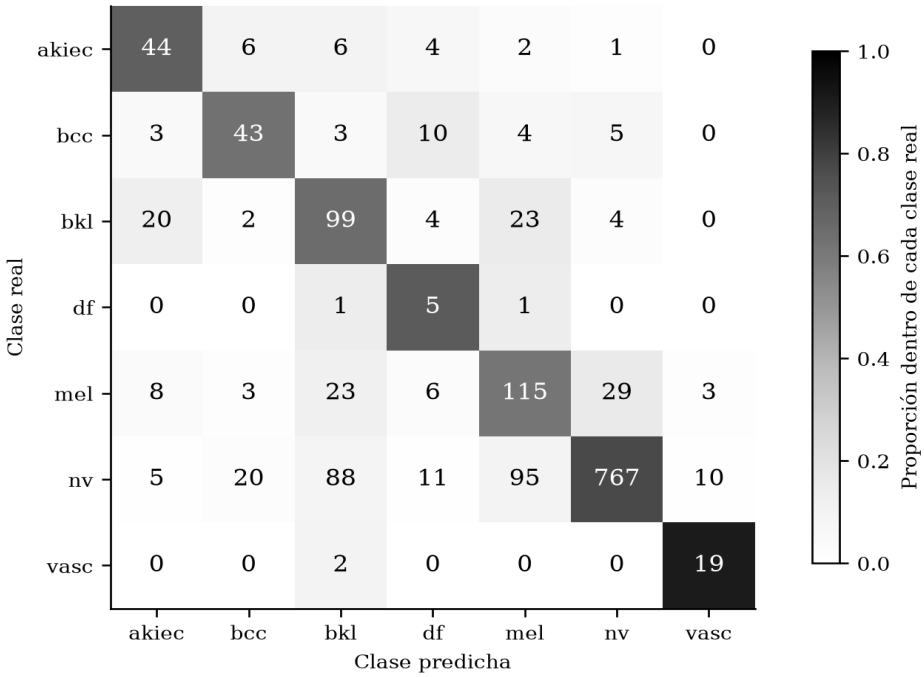


Figura 3. Matriz de confusión de las siete categorías, en el mismo orden de la tabla. Cada celda indica el número de imágenes y su intensidad de gris refleja la proporción dentro de la clase real.

El modelo identificó correctamente **115 de los 187 melanomas (61,50 %)**. Los 72 restantes se asignaron a otras categorías, sobre todo nv (29 imágenes) y bkl (23); al mismo tiempo, 95 nevus se clasificaron como melanoma. El caso de df también requiere cautela: acertar cinco de sus siete ejemplos produjo una sensibilidad de 71,43 %, pero hubo 40 predicciones de esa clase y la precisión fue de apenas 12,50 %. Con tan pocos ejemplos, una variación pequeña en los aciertos cambiaría considerablemente la estimación.

6. Conclusiones, límites y reproducibilidad

La comparación permitió observar que el efecto de cada estrategia depende del aspecto del rendimiento que se considere. La CNN con pesos de clase mejoró su balanced accuracy de validación y, en prueba, fue el modelo binario con mayor sensibilidad. ResNet50 ofreció un mejor equilibrio según el F1 macro y una mayor exactitud global, aunque dejó más lesiones positivas sin detectar. En siete clases, ajustar el último bloque mejoró la validación frente a entrenar solo el clasificador, pero los errores de melanoma y el F1 macro de prueba muestran que todavía hay dificultades relevantes.

La meta exploratoria de sensibilidad superior al 85 % se cumplió con la CNN únicamente al incluir el melanoma en el grupo binario positivo. Al exigir su identificación específica entre las siete categorías, la sensibilidad quedó en 61,50 %. Esta diferencia es central para interpretar el alcance del proyecto y evita trasladar un resultado favorable de la tarea binaria a una afirmación sobre capacidad diagnóstica.

Límites y trabajo futuro

Las conclusiones se apoyan en una sola partición y una semilla, sin validación cruzada, intervalos de confianza ni evaluación externa. A ello se suman la separación por lesión, que no asegura independencia entre pacientes, y el reducido número de ejemplos de algunas categorías. Los presupuestos de entrenamiento fueron limitados y no se estudiaron calibración, umbrales clínicos ni muestreo ponderado. Una continuación del trabajo debería medir la variabilidad entre ejecuciones y confirmar el desempeño con datos independientes, evitando utilizar este conjunto de prueba, ya consultado, para ajustar nuevas decisiones.

Cómo reproducir y rastrear las cifras

La reproducción sigue el orden de los notebooks 01 a 05 con el entorno y los datos descritos. El segundo notebook establece las particiones; el tercero y el cuarto registran las selecciones binarias, y el quinto desarrolla las dos etapas multiclase con la arquitectura elegida. Las cifras de este informe corresponden a las ejecuciones completas identificadas como **local_v2**. Para nuevos experimentos se debe conservar la separación entre las decisiones de validación y la evaluación final.

La trazabilidad se conserva en **data/splits/manifest_local.json**, en la auditoría **image_audit.csv** y en los historiales de **results/full/local_v2/**. Las subcarpetas **final_test/{baseline,transfer,multiclass}/** contienen las métricas y predicciones finales. La revisión comprobó las etiquetas, probabilidades y matrices de las mismas 1.494 imágenes, así como la correspondencia entre selecciones y checkpoints. Los notebooks publicados conservan las tablas y figuras de las ejecuciones.

Código y salidas auditadas: MiltonCuervo/ham10000-skin-lesion-classification · commit e3438f9. La propuesta original se conserva como ENTREGA1.PDF.

Referencias

[1] Tschandl, P.; Rosendahl, C.; Kittler, H. (2018). The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions. Scientific Data, 5, 180161. doi:10.1038/sdata.2018.161.

[2] Simonyan, K.; Zisserman, A. (2015). Very Deep Convolutional Networks for Large-Scale Image Recognition. ICLR. arXiv:1409.1556.

[3] He, K.; Zhang, X.; Ren, S.; Sun, J. (2016). Deep Residual Learning for Image Recognition. CVPR, 770–778. doi:10.1109/CVPR.2016.90.

[4] Tan, M.; Le, Q. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML, PMLR 97, 6105–6114. proceedings.mlr.press/v97/tan19a.html.

[5] Kaggle, kmader. Skin Cancer MNIST: HAM10000, distribución del conjunto. kaggle.com/datasets/kmader/skin-cancer-mnist-ham10000. Consulta: 13/11/2026.